



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

並行程序设计期末实验报告

SIMD 编程实验

顏國庄

年級：2024 級

專業：計算機科學與技術

指導教師：王剛

2026 年 5 月 10 日

目录

一、 串行算法运行测试	1
二、 并行算法设计	1
(一) 对 gkh.cpp 文件的处理	1
(二) 对 bidiagonalization.cpp 文件的处理	2
(三) 优化结果	4
三、 性能对比	4
(一) 不开启编译优化的情况下	4
(二) O1 优化情况下	5
(三) O2 优化情况下	6
四、 编译选项对加速比的影响	7
(一) g++ 编译器优化策略调研	7
(二) 对实验结果的分析	8
(三) 对加速比的影响	8
五、 perf 分析	9
六、 代码链接	9
七、 大模型辅助问答记录	10

一、 串行算法运行测试

运行串行算法，返回结果，可以看到程序正常运行并返回结果，在此基础上我们开始我们的实验。

```
=====
随机种子基值: 20260409
总上二对角化耗时(ms): 8589.2
总GKH迭代耗时(ms): 32042.9
通过: 5 / 5

Authorized users only. All activities may be monitored and reported.
```

图 1: 串行算法测试 (O2 优化条件下)

二、 并行算法设计

(一) 对 gkh.cpp 文件的处理

在该阶段，核心操作是对矩阵的特定行进行线性组合更新。具体实现是基于 NEON128 位指令集，其一次指令处理一个 128 位向量，也就是两个 64 位数据。由于 SVD 计算中涉及到极高精度的浮点运算，底层数据类型均为 double (64 位)，因此一个 128 位的 NEON 寄存器一次可精确打包并处理 2 个双精度浮点数。核心代码实现如下：

```
1  static void apply_left_rows(Matrix &M, int r0, int r1, double c, double
2      s)
3  {
4      const int cols = M.cols();
5      double* row0 = &M.at(r0, 0);
6      double* row1 = &M.at(r1, 0);
7
8      float64x2_t vc = vdupq_n_f64(c);
9      float64x2_t vs = vdupq_n_f64(s);
10     float64x2_t vns = vdupq_n_f64(-s);
11
12     int j = 0;
13     for (; j + 2 <= cols; j += 2)
14     {
15         float64x2_t va = vld1q_f64(row0 + j); // [a0, a1]
16         float64x2_t vb = vld1q_f64(row1 + j); // [b0, b1]
17
18         // new_row0 = c*a + s*b
19         float64x2_t new_a = vfmaq_f64(vmulq_f64(vs, vb), vc, va);
20
21         // new_row1 = -s*a + c*b
22         float64x2_t new_b = vfmaq_f64(vmulq_f64(vc, vb), vns, va);
23
24         vst1q_f64(row0 + j, new_a);
```

```

24     vst1q_f64(row1 + j, new_b);
25     .....
26 }

```

并行版本即每次处理一个 128 位向量（对应 2 个 64 位双精度浮点数），对应的运算操作也应该适配于 128 位向量，`vdupq_n_f64` 对应的是将标量常数广播至整个向量寄存器，`vld1q_f64` 对应的是对连续内存地址的向量加载运算；

核心的线性组合旨在实现 128 位向量的融合乘加操作，即 2 个双精度浮点数的并行乘加计算，只需将对应向量进行相乘，再加上另一个向量的结果合并起来即可。`vfmaq_f64` 是融合乘加操作，第一个参数为累加基准向量，第二和第三个参数为需要相乘的两个向量，`vmulq_f64` 对应了基础的向量乘法操作；

在修改完代码后，进行测试运行，发现总.cpp 文件的处理耗时有时低于优化前，有时却高于优化前。原因可能是 1. 该函数操作的矩阵本身较小，优化收益微乎其微，而 NEON 引入了额外开销，比如指令更多、启动开销更大。或是 2. 服务器集群不稳定，导致测试运行的结果也会有差异。

（二） 对 bidiagonalization.cpp 文件的处理

左侧 householder 变换原代码中按列遍历引发严重跨步访问，不符合底层矩阵行主序的物理特性，我利用循环交换改为了顺着内存地址的连续行向读取。

```

1  for (int i = 0; i < rows; ++i)
2  {
3      float64x2_t scale = vdupq_n_f64(beta * v[i]);
4
5      int j = 0;
6      for (; j + 2 <= cols; j += 2)
7      {
8          double* bptr = &B.at(k+i, k+j);
9
10         float64x2_t bv = vld1q_f64(bptr);
11         float64x2_t wv = vld1q_f64(&w[j]);
12
13         bv = vfmsq_f64(bv, scale, wv);
14
15         vst1q_f64(bptr, bv);
16     }
17     // 尾部标量兜底处理 ...
18 }

```

通过 `vdupq_n_f64` 指令将内层循环中保持不变的标量常数提前广播至 128 位向量寄存器。其次在内层连续内存遍历中，使用 `vld1q_f64` 指令实现双精度浮点数的高效批量加载。在核心的运算环节摒弃了乘法与加减法的分离执行，采用了 `vfmsq_f64` 和 `vfmaq_f64` 融合指令，在单一时钟周期内完成向量的乘累加/减操作。最后剩余的部分，由于无法存到寄存器中，使用标量进行处理。

右侧 Householder 变换的 SIMD 优化继承了左侧变换中高度成熟的指令调度策略（如 `vfmaq_f64` 融合乘加）。但得益于右乘操作天然契合行主序内存的特性，右侧优化省去了复杂的循环重排开销，核心代码：

```

1 // 第一步:  $w[i] = \sum_j B[k+i, k+1+j] * v[j]$ 
2 // 外i内j, B行连续访问
3 std::vector<double> w(rows, 0.0);
4
5 for (int i = 0; i < rows; ++i)
6 {
7     float64x2_t acc = vdupq_n_f64(0.0);
8     int j = 0;
9     for (; j + 2 <= cols; j += 2)
10    {
11        float64x2_t bv = vld1q_f64(&B.at(k+i, k+1+j)); // B行连续
12        float64x2_t vv = vld1q_f64(&v[j]);
13        acc = vfmaq_f64(acc, bv, vv);
14    }
15    // 水平规约
16    w[i] = vgetq_lane_f64(acc, 0) + vgetq_lane_f64(acc, 1);
17    for (; j < cols; ++j)
18        w[i] += B.at(k+i, k+1+j) * v[j];
19 }
20
21 // 第二步:  $B[k+i, k+1+j] \leftarrow \text{beta} * w[i] * v[j]$ 
22 // 外i内j, B行连续访问
23 for (int i = 0; i < rows; ++i)
24 {
25     float64x2_t scale = vdupq_n_f64(beta * w[i]); // 广播 beta*w[i]
26     int j = 0;
27     for (; j + 2 <= cols; j += 2)
28     {
29         double* bptr = &B.at(k+i, k+1+j);
30         float64x2_t bv = vld1q_f64(bptr);
31         float64x2_t vv = vld1q_f64(&v[j]);
32         bv = vfmsq_f64(bv, scale, vv); //  $B \leftarrow \text{scale} * v$ 
33         vst1q_f64(bptr, bv);
34     }
35     for (; j < cols; ++j)
36         B.at(k+i, k+1+j) -= beta * w[i] * v[j];
37     .....
38 }

```

上述代码展示了右侧变换更新主矩阵 B 的核心向量化逻辑。第三步与第四步中更新正交矩阵 V 的算法操作与此同构, 在此省略。

(三) 优化结果

```
=====
随机种子基值: 20260409
总上二对角化耗时(ms): 4324.94
总GKH迭代耗时(ms): 27955.7
通过: 5 / 5
```

图 2: 并行优化结果 (O2 优化条件下)

三、性能对比

(一) 不开启编译优化的情况下

```
=====
随机种子基值: 20260409
总上二对角化耗时(ms): 40239.7
总GKH迭代耗时(ms): 126190
通过: 5 / 5
```

图 3: 串行算法

```
=====
随机种子基值: 20260409
总上二对角化耗时(ms): 39159.1
总GKH迭代耗时(ms): 134443
通过: 5 / 5
```

图 4: 并行算法

我们发现并行算法总 GKH 迭代耗时大于串行算法。并行性能不及串行的原因可能有两方面：一是并行算法将数据切割分配给多线程时，导致了不连续的内存访问（如跨步访问），使得缓存命中率下降，而串行算法的连续访问对硬件预取更加友好；二是由于缺乏编译器优化，代码中大量使用的 NEON 内部函数未能实现内联展开，带来了极大的额外调用耗时。通过修改种子重新运行，得到不同种子的串行算法时间和并行算法时间。

表 1: 不开启编译优化的情况下 (09 种子为重新运行结果)

种子	串行对角化时间	并行对角化时间	串行迭代时间	并行迭代时间
20260408	38408.7ms	41847.1ms	122000ms	104415ms
20260409	38285.4ms	39413.5ms	140800ms	123760ms
20260410	39814.9ms	40026.4ms	137778ms	119478ms
20260411	40173.3ms	37187.3ms	145597ms	123960ms
20260412	39382.1ms	37524.7ms	139063ms	123641ms

在未开启编译优化的情况下, 并行算法的时间有些会长于串行算法。猜测是由于上面提到的情况。

(二) O1 优化情况下

```
=====
随机种子基值: 20260409
总上二对角化耗时(ms): 15068.6
总GKH迭代耗时(ms): 24202.2
通过: 5 / 5
```

图 5: 串行算法

```
=====
随机种子基值: 20260409
总上二对角化耗时(ms): 4098.76
总GKH迭代耗时(ms): 30214.2
通过: 5 / 5
```

图 6: 并行算法

在 O1 优化的情况下, 并行对角化时间显著小于串行时间, 迭代时间并行大多数小于串行代码。可以说明在 O1 编译优化的情况下, 并行算法大概率会优于串行算法。

表 2: O1 编译优化的情况下 (09 种子为重新运行结果)

种子	串行对角化时间	并行对角化时间	串行迭代时间	并行迭代时间
20260408	15600.0ms	5689.59ms	33074.7ms	32204.6ms
20260409	14964.1ms	5889.27ms	43745.2ms	31321.9ms
20260410	14865.7ms	5830.25ms	24488.5ms	32129.9ms
20260411	14851.7ms	4304.55ms	32874.3ms	31378.9ms
20260412	16306.8ms	5772.49ms	34728.2ms	22356.7ms

(三) O2 优化情况下

```

=====
随机种子基值: 20260409
总上二对角化耗时(ms): 7912.8
总GKH迭代耗时(ms): 37670.6
通过: 5 / 5

```

图 7: 串行算法

```

=====
随机种子基值: 20260409
总上二对角化耗时(ms): 6063.84
总GKH迭代耗时(ms): 27494.2
通过: 5 / 5

```

图 8: 并行算法

表 3: O2 编译优化的情况下 (09 种子为重新运行结果)

种子	串行对角化时间	并行对角化时间	串行迭代时间	并行迭代时间
20260408	7961.35ms	5729.66ms	30603.0ms	36851.5ms
20260409	7721.52ms	5667.28ms	39816.1ms	33361.0ms
20260410	10201.2ms	4167.87ms	34828.7ms	28783.2ms
20260411	8061.14ms	5689.76ms	38095.1ms	29755.1ms
20260412	9349.45ms	4197.78ms	28818.5ms	22760.1ms

在 O2 编译优化的情况下, 也类似于 O1, 并行算法大部分优于串行算法。但是可以看出, 在

多数情况下，O2 编译优化的加速比低于 O1 优化的加速比，这可能与 O2 优化也对串行算法优化更多，使加速比变小。

四、 编译选项对加速比的影响

(一) g++ 编译器优化策略调研

-O0 优化选项

-O0 是无优化策略，是编译器的默认选项。编译速度最快，调试最容易，适用于开发和调试阶段。

-O1 优化选项

-O1 旨在不显著增加编译时间的情况下缩减代码大小并提升运行速度。-O1 的一些优化技术有：

函数内联：将短小的函数调用直接替换为函数体代码，减小函数调用的栈操作开销。

死代码消除：移除那些永远不会执行或执行结果从未被使用的指令。

常量折叠与传播：在编译阶段计算 $2 + 3$ 这种确定值，并将变量替换为其已知的常量值，减少运行时计算。

公共子表达式消除：识别重复计算的部分（如 $a*b$ 出现多次），将其存入临时寄存器，避免重复运算。

虽然 -O1 去除了大量冗余逻辑，但它对控制流的重构非常保守。对于大量使用内联函数的向量化代码，-O1 通常不会进行彻底的函数内联展开。这意味着底层操作仍可能引发函数调用栈的保存与恢复开销。

-O2 优化选项

-O2 是兼顾目标文件大小与执行速度的推荐级别。它包含了 -O1 的所有优化，并额外启用了大量不牺牲空间换取时间的进阶分析。核心优化机制有：

循环优化：包括循环展开、循环交换等，减少循环的开销并提高缓存的利用率。

指令调度：根据 CPU 流水线特性重新排列指令顺序，减少执行停顿。

控制流优化：通过重新安排跳转逻辑来减少分支预测失败带来的开销。

-O2 能够将 C++ 代码中包装的 SIMD 操作直接在调用处替换为原生的底层 ARM 汇编指令，减少了指令级并行中的上下文切换开销。

-O3 优化选项

-O3 会为了性能而不惜增加二进制文件的大小。核心技术有：

自动向量化：将普通的标量计算转换为 SIMD 指令，使 CPU 能在一个时钟周期内处理多个数据。

大规模循环展开；

对更大的函数进行内联展开；

在本实验中 -O3 的效果并不一定优于 -O2。大规模的循环展开会导致生成的机器码数量增多，极易超出 CPU 指令缓存的容量，从而引发指令缓存未命中。

特殊目的优化

-Os：在 -O2 的基础上屏蔽掉会增大代码体积的优化（如内联和展开），适用于存储空间受限的设备。

-Ofast：开启 -O3 的所有优化，并加入非标准数学优化，可能导致浮点数精度出现微小偏差。

-flto：链接时优化。传统的编译是以单个文件为单位的，通过开启链接时优化，编译器能够在最终的链接阶段跨越单个代码文件的边界，获取整个工程的完整上下文，从而执行跨文件的函

数内联与死代码消除。

(二) 对实验结果的分析

我生成了并行 O1 的优化报告，发现有很长一部分出现了 inline 的标志。通过查看，发现编译器对很多 NEON 指令进行了内联展开，减少函数调用的开销，这正好说明了并行算法运行短于串行算法的原因。另外这也解释了为什么在未开编译优化的情况下，并行算法有时候会比串行算法慢，原因是我们使用了很多次 NEON 指令函数，调用了很多次，导致运行时间会比较长。报告片段如下：

```
bidiagonalization.cpp:292:29: optimized: Inlining float64x2_t vfmsq_f64(
bidiagonalization.cpp:291:41: optimized: Inlining float64x2_t vld1q_f64(
bidiagonalization.cpp:290:41: optimized: Inlining float64x2_t vld1q_f64(
bidiagonalization.cpp:285:40: optimized: Inlining float64x2_t vdupq_n_f6
bidiagonalization.cpp:274:56: optimized: Inlining float64_t vgetq_lane_f
bidiagonalization.cpp:274:31: optimized: Inlining float64_t vgetq_lane_f
bidiagonalization.cpp:272:28: optimized: Inlining float64x2_t vfmaq_f64(
bidiagonalization.cpp:271:41: optimized: Inlining float64x2_t vld1q_f64(
bidiagonalization.cpp:270:41: optimized: Inlining float64x2_t vld1q_f64(
bidiagonalization.cpp:266:38: optimized: Inlining float64x2_t vdupq_n_f6
bidiagonalization.cpp:252:22: optimized: Inlining void vst1q_f64(float64
bidiagonalization.cpp:251:27: optimized: Inlining float64x2_t vfmsq_f64(
bidiagonalization.cpp:250:41: optimized: Inlining float64x2_t vld1q_f64(
bidiagonalization.cpp:249:41: optimized: Inlining float64x2_t vld1q_f64(
bidiagonalization.cpp:244:40: optimized: Inlining float64x2_t vdupq_n_f6
```

图 9: 并行 O1 优化报告片段

另外生成串行 O1 的优化报告，我发现除了没有对于 NEON 指令的内联展开外，其他部分大致相同。在 O1 优化情况下，消除了 NEON 函数调用的开销，使得并行算法优于串行算法。

然后，我生成了并行 O2 的优化报告，发现大多数也是函数内联展开，同时也有循环展开的代码。相较于 O1，运行速度也快了一点，代码片段如下：

```
/usr/include/c++/10.3.1/bits/stl_vector.h:683:7: optimized: Inlined std::vector_base<Tp, _Alloc>::_M_vector_base() [wi
bidiagonalization.cpp:295:18: optimized: loop with 1 iterations completely unrolled (header execution count 20339794)
bidiagonalization.cpp:254:18: optimized: loop with 1 iterations completely unrolled (header execution count 20352383)
bidiagonalization.cpp:167:18: optimized: loop with 1 iterations completely unrolled (header execution count 17233444)
bidiagonalization.cpp:125:18: optimized: loop with 1 iterations completely unrolled (header execution count 17244111)
bidiagonalization.cpp:105:18: optimized: loop with 1 iterations completely unrolled (header execution count 17244111)
bidiagonalization.cpp:302:31: optimized: loop 33 distributed: split to 0 loops and 1 library calls.
```

图 10: 并行 O2 优化报告片段

同时通过阅读报告末尾并查阅资料，发现了 O2 优化的一个机制：循环习语识别。编译器如果发现一部分我们在底层写的 for 循环其实是在做单纯的内存拷贝或填充，编译器就会将这些循环替换成库函数调用。

```
/usr/include/c++/10.3.1/bits/stl_algobase.h:872:22: optimized: Loop 17 distributed: split to 0 loops and 1 library calls.
/usr/include/c++/10.3.1/bits/stl_algobase.h:872:22: optimized: Loop 20 distributed: split to 0 loops and 1 library calls.
```

图 11: 报告末尾片段

(三) 对加速比的影响

基于实验结果和资料查询，在不开编译优化的情况下，并行算法可能因为大量调用函数而差于串行算法，加速比较低；当开了 O1 编译优化后，并行算法会优于串行算法，加速比会大于 1；

当使用更高级别的编译优化时，如果并行算法存在可优化逻辑，那么可能会使加速比增高，否则如果无明显优化，加速比会因串行算法运行时间降低而减小。比如在我们的实验中，算法在 O2 编译情况下加速比低于 O1。

五、 perf 分析

表 4: 基于 perf 性能分析

核心性能指标	串行	并行
总运行时间	125.27 s	120.90 s
执行指令数	1189.1 亿条	1206.8 亿条
消耗周期数	3190.7 亿个	3073.3 亿个 (-3.6%)
IPC	0.37	0.39
L1 缓存缺失	32.5 亿次 (7.3%)	29.7 亿次 (7.1%)
分支预测失误	2315 万次	2082 万次

我们依次来分析这些指标，首先是运行时间，并行时间少于串行时间，这是我们预料之中的，尽管差距看似不大，但考虑到这涵盖了框架开销，核心函数的加速是显著的。

接着来看总指令数，十分奇怪串行指令数为 1189.1 亿条，并行指令数反而上升至 1206.8 亿条。可以看到，并行指令数略多于串行指令数。可能的原因是在并行算法中，由于我们需要对行主序矩阵进行列操作，不得不使用多条标量加载和向量插入操作来手动拼接 128 位寄存器数据。这些为了满足 SIMD 对齐而产生的额外数据装配指令，抵消了融合乘加减少的冗余计算，进而增加了总指令数。

然后是 IPC，并行版本的 IPC 是 0.39，高于串行版本的 0.37，说明并行算法每条指令的平均执行周期更短、流水线效率更高。这里的原因是，虽然程序面临跨步访存带来的内存受限问题，但通过 perf 的 L1 缓存缺失数据可知，并行版本的缓存缺失从 32.5 亿次降至 29.7 亿次。我们在内层循环一次性预取 4 行数据的策略，减少了等待主存访问的时间，抵消了部分访存延迟的增加。

相较于串行算法，并行算法的指令数量虽多，但由于流水线效率高且缓存缺失变少，消耗的总时钟周期下降，使得总的运行时间依然实现了缩短。

还有一个指标是分支预测失误率，串行算法的分支预测失误次数约为 2315 万次，并行算法降至 2082 万次，并行分支预测更优。这主要是因为我们在代码中直接减少了外层条件分支的判断次数，优化了预测逻辑实现，大幅降低了昂贵的流水线冲刷代价。

总的来说，并行算法相较于串行算法来说，确实实现了一定的优化。但是由于更多的时间依然浪费在行主序导致的离散内存访问上，访存受限仍然是主要瓶颈，可能存在更好的并行优化结构（例如采用分块算法或转为列主序），仍有极大的优化空间。

六、 代码链接

[点击访问 github 主页](#)

七、 大模型辅助问答记录

Q1: 我想使用 cache 加速 matrix 类的矩阵乘法, 这个想法怎么样?

A1: 如果 `operator*` 只在验证结果时被调用, 且这部分代码根本不在计时器 (Timer) 的统计范围内。那你就算把它优化到光速, 你的“并行加速比”也不会提升哪怕 0.01! 这就属于典型的“在非瓶颈区域做无用功”。

Q2: 我的并行算法指令数多于串行算法指令数, 可能是什么原因?

A2: 这种“指令反向膨胀”现象揭示了底层微架构的物理限制: 在行主序矩阵中对列进行操作时 (跨步访存), 由于 ARM 架构缺乏原生的硬件级 Gather (离散收集) 指令, 我们不得不使用多条标量加载 (LDR) 和向量插入 (INS) 指令来手动拼接 128 位的向量寄存器。正是这部分冗余的“数据装配指令”, 抵消了 SIMD 融合乘加 (FMA) 减少的计算指令, 导致总指令数上升。

NIJUB